

Personal Growth Engine

A live, automated GTM pipeline — built as a proof of concept for modern growth infrastructure.

Author: Julian Jais
Role: GTM Architect • Growth Ops
Website: julianjais.com
Published: May 2026
Version: 1.0

The idea: invert the recruiting funnel.

Instead of sending cold applications, I built the same GTM systems I architect for companies — and deployed them on my own portfolio website. Every recruiter who visits julianjais.com becomes a lead in a live, automated pipeline.

The system captures first-party behavioral data, enriches every contact with company intelligence, and sends an AI-personalised reply within minutes — without any manual intervention. This is not a demo environment. It runs in production, on real traffic, with real data flowing through every layer.

4

PHASES COMPLETED

8

TOOLS INTEGRATED

0

MANUAL STEPS

6 wks

TIME TO BUILD

EU

DATA LOCATION (GDPR)

< €30

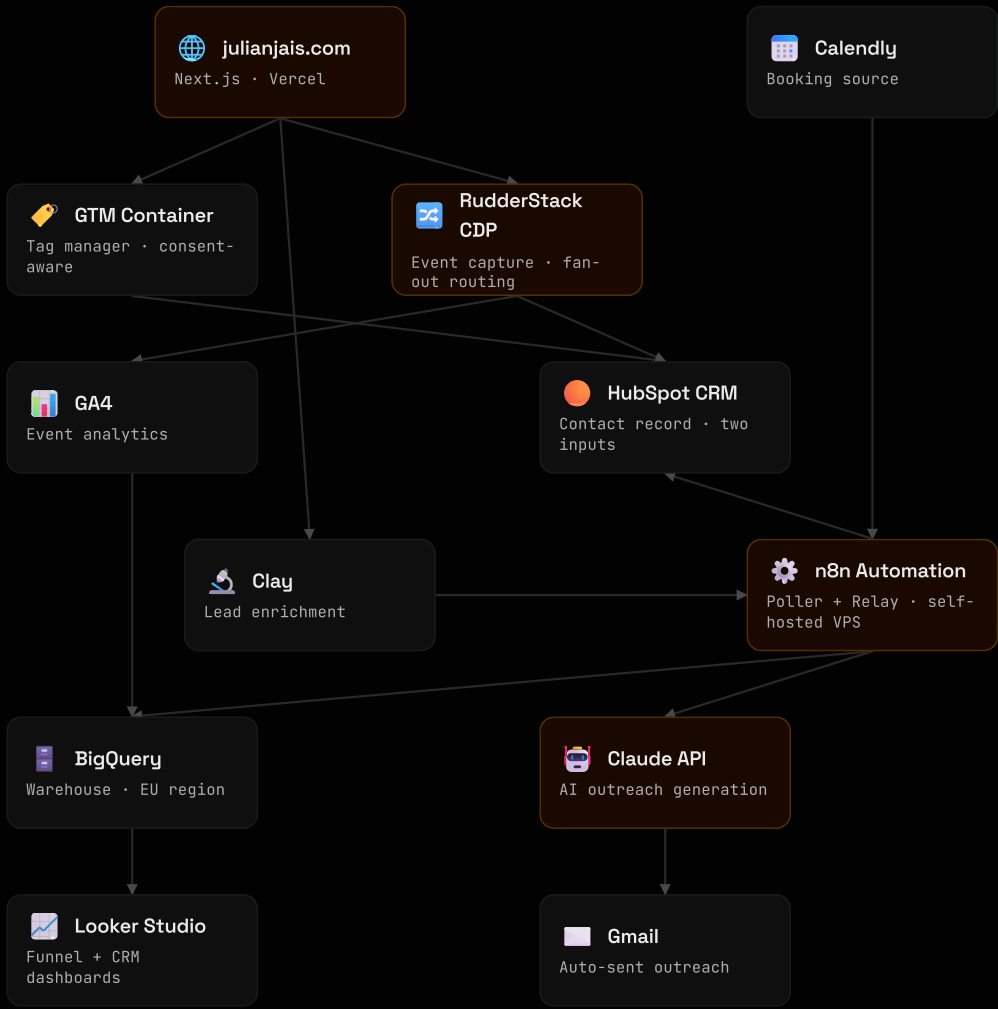
STACK COST / MONTH

What this demonstrates

- End-to-end GTM architecture design and implementation
- First-party data infrastructure with consent management
- CDP fan-out routing (single event → multiple destinations)
- AI-powered workflow automation without code templates
- Server-side tracking to preserve data quality
- Cloud data warehousing in a GDPR-compliant setup

How the pipeline works

Two entry points feed a single automation backbone. Every layer was chosen for integration depth and real-world applicability — not resume padding.



Build phases 1 & 2

01

GTM + RudderStack

Tracking Foundation

Set up the complete event-tracking stack from scratch. Google Tag Manager was configured with Consent Mode v2 defaults — all analytics denied until explicit user consent. CookieYes handles the consent banner and fires a custom event on update.

RudderStack acts as the CDP layer: a single JavaScript SDK in the browser captures all events and fans them out server-side to GA4, HubSpot, and BigQuery simultaneously. No GA4 script in the browser — tracking happens entirely through RudderStack's backend.

Custom DataLayer events:

- page_viewed (past tense — GA4 reserves 'page_view')
- cta_clicked (all CTA interactions)
- section_viewed (scroll depth via IntersectionObserver)
- contact_form_submitted
- meeting_booked (Calendly confirmation)

GTM

RudderStack

CookieYes

GA4

DataLayer

Consent Mode v2

Next.js

02

HubSpot & Calendly

CRM Integration

Connected RudderStack to HubSpot as a destination. Every identify() call — triggered on contact form submission — creates or updates a contact record in HubSpot CRM automatically. No manual data entry, no Zapier middleman.

Calendly was embedded directly in the Next.js page using the official widget. A postMessage listener captures the calendly.event_scheduled browser event and fires a meeting_booked track event into the RudderStack pipeline — landing in both GA4 and HubSpot simultaneously.

Architecture decision: using RudderStack's native HubSpot destination instead of a GTM-triggered HubSpot pixel keeps the CRM integration consent-aware by default. The pixel only loads after Analytics consent is granted.

HubSpot Free

Calendly

postMessage API

RudderStack Destinations

CRM Identify

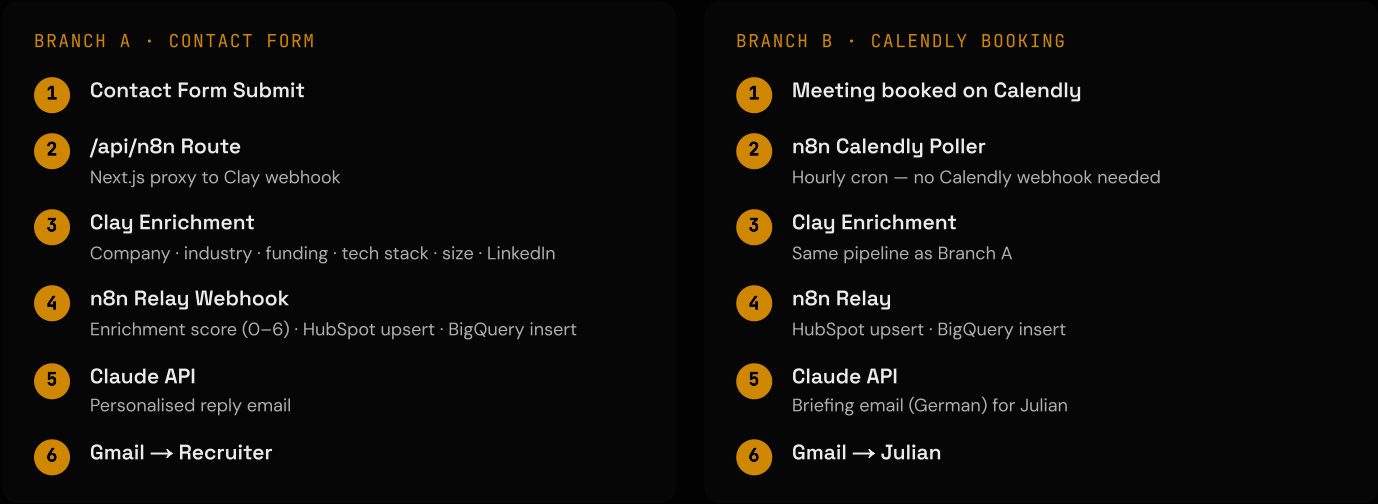
KEY CHALLENGE SOLVED • PHASE 1

CookieYes fires 'cookieyes_consent_update' (underscore) in current versions — the listener was built for 'cookieyes-consent-update' (hyphen). Fixed with a robust multi-event listener on both document and window, supporting all known event name variants.

n8n Workflow

Enrichment & AI Outreach

The automation backbone — the layer that turns a form submission into a personalised conversation within minutes.



Key decisions

- Prompt engineering: Claude was initially writing "Hi [Name]," literally as a placeholder. Fixed by pre-computing firstName in the Code node and using an explicit system prompt: "NEVER use placeholders. Open with exactly: Hi {firstName},"
- Enrichment score: a 0–6 integer calculated from the number of Clay fields successfully populated (industry, company_size, funding_stage, tech_stack, linkedin_url, job_title). Written to BigQuery for lead quality analysis.

n8n (self-hosted)

Clay Free

Claude API (Sonnet)

Gmail

Hostinger VPS

Webhooks

Calendly Poller

BigQuery + Analytics

Data Infrastructure

GA4 → BigQuery

Activated GA4's native BigQuery linking — no code required. Daily exports land in dataset analytics_534559293 as events_YYYYMMDD tables, partitioned by date. Export runs in the EU region (GDPR compliant). Streaming export was deliberately disabled to stay within the free tier.

Since GA4 receives data via RudderStack's server-side pipeline (no GA4 browser script), there are no GA4 cookies on the website — a privacy architecture advantage.

Available tables (as of Phase 4 close): events_20260518 through events_20260521 ✓ — page_viewed fix deployed 20260522.

n8n → BigQuery

Two blockers required creative workarounds.

Blocker 1 — Service account keys

Google's default policy iam.disableServiceAccountKeyCreation was active — no JSON keys could be created on a personal Gmail account. Solution: OAuth2 Client ID instead of service account. Credentials stored in n8n, tokens refreshed automatically.

Blocker 2 — Native BigQuery node bug

n8n BigQuery node v2.1 threw a validator error ('timestamp is required') despite correct input. Solution: replaced with a direct HTTP Request node calling the BigQuery REST API insertAll endpoint with the same OAuth2 credential.

SCHEMA · CRM_DATA.CONTACTS (EUROPE-WEST3)

timestamp, source, email, first_name, last_name, company, job_title, linkedin_url, industry, company_size, funding_stage, tech_stack, hubspot_contact_id, enrichment_score

Verification: 3 test rows written, all with enrichment_score 4/6.

BigQuery

GA4 Native Export

OAuth2

n8n HTTP Request

europe-west3

Looker Studio

What I learned

1 Consent architecture comes first, not last.

Building consent mode into the tracking foundation from day one — instead of retrofitting it — saved significant rework. Every downstream tool inherits the consent state automatically through RudderStack.

2 Workarounds are architecture decisions.

Both major blockers (OAuth2 vs service account keys, HTTP node vs native BigQuery node) produced more resilient solutions than the "happy path" would have. The direct REST API approach is actually more transparent and debuggable than the native node.

3 Prompt engineering is a real discipline.

Getting Claude to write "Hi Julian," instead of "Hi [Name]," required understanding how language models handle instructions vs. data. Pre-computing variables and explicit system-level constraints are non-negotiable in production AI workflows.

4 Server-side > client-side for data quality.

Routing GA4 data through RudderStack's backend eliminated ad-blocker interference, reduced browser cookie footprint, and simplified consent management — all simultaneously.

What I would do differently

- Use a proper data catalogue from day one (BigQuery table descriptions, schema documentation)
- Set up staging and production environments for n8n workflows before going live
- Implement error alerting on the n8n relay webhook earlier — silent failures are hard to debug
- Consider RudderStack Cloud (managed) vs self-hosted for production use cases with SLA requirements

Full tool reference

TOOL	CATEGORY	PURPOSE	HOSTING	COST/MO
Next.js	Frontend	Portfolio website	Vercel (EU CDN)	Free
Vercel	Hosting	Deployment & CDN	USA / Global	Free
GitHub	Version Control	Source code management	USA	Free
GTM	Tag Mgmt	Tag orchestration	Google (EU)	Free
CookieYes	Consent	Cookie consent banner	Cloud	Free
RudderStack	CDP	Event capture & fan-out	Cloud (USA)	Free
GA4	Analytics	Behavioural analytics	Google (USA)	Free
HubSpot	CRM	Contact management	Cloud (USA)	Free
Calendly	Scheduling	Meeting booking	Cloud (USA)	Free
n8n	Automation	Workflow orchestration	Hostinger VPS	~€5
Clay	Enrichment	Lead data enrichment	Cloud (USA)	Free
Claude API	AI	Personalised outreach	Anthropic (USA)	Pay-per-use
BigQuery	Data Warehouse	Event & CRM storage	Google (EU-W3)	Free tier
Looker Studio	BI	Analytics dashboards	Google	Free

TOTAL

~€5-15 / month

depending on Claude API usage volume